FALCON 9 FIRST-STAGE LANDING

Applied Data Science Capstone | Evidence-based project report and presentation narrative

Research question: How do mission, vehicle, launch site, and flight-history attributes relate to first-stage landing success? This report includes independently computed descriptive statistics from an open educational dataset. Predictive model scores and application screenshots are intentionally not claimed without execution evidence.

Study scope: 90 dated Falcon 9 launch records, 2010–2020. Public educational data reproduced from Abhigyan-RA/SpaceX-capstone/dataset_part_2.csv. Dataset provenance should be disclosed on submission. Data do not represent all SpaceX missions through 2026.

Project repository: Not supplied or connected for this capstone. A dedicated public GitHub repository with notebooks and scripts must be linked prior to final submission.

Executive summary

Of 90 mission rows, **60 have Class = 1 (66.7%)**. Observed success varies by launch site (60.0%–77.3%), is higher among reused boosters (78.4% versus 58.5% for not reused), and shows a pronounced relationship with grid fins (82.9% with grid fins versus 10.0% without). These are associations rather than causal estimates. Longitudinal trends also fluctuate as Falcon 9 configurations and mission mix change.

Key recommendation: Assess class balance and potential time-dependent leakage before presenting any model as a forward-looking predictor. Split chronologically for an additional robustness check, and compare models against a majority-class baseline (66.7%).

1. Introduction and problem statement

SpaceX first-stage reuse may alter launch economics. For a launch-services analyst, estimating landing outcomes can support preliminary scenario analysis, although operational and financial decisions require much richer data. Our target is the historical binary variable **Class**, where 1 indicates an outcome labeled successful in the supplied educational data.

Goals: collect launch data through SpaceX API and public web tables; harmonize flight features; examine geographic, mission and time patterns; build SQL summaries and visualizations; create a Folium map and a Plotly Dash app; then compare supervised classifiers on held-out observations.

2. Data collection: SpaceX API and web scraping

API collection process

Workflow for a reproducible original project: call the SpaceX public launches endpoint; normalize the JSON response; enrich rocket, launchpad, and landing-pad identifiers through API lookups; create columns for launch date, booster version, payload mass, orbit, flight count, and landing outcomes; save a versioned raw CSV and record retrieval time. Store request status, schema, and exception handling in the notebook.

Evidence status: The source dataset contains these API-style fields; no newly executed API calls or screenshots were provided, so this report does not claim that its author performed those calls.

Web scraping process

Scrape an appropriately attributed Falcon 9 launch table using pandas.read_html or BeautifulSoup, clean multirow headings, convert dates and payload units, standardize categorical outcomes, and validate row counts. Reconcile API and scraped data via flight date and flight identifier where possible; do not assume that two sources share identical landing success semantics.

Suggested data lineage: Source page/API → downloaded raw file → normalization → deduplication → merged mission table → analytic extracts. Include both notebook and GitHub URLs on the submitted slides.

3. Data wrangling methodology

Parse Date as a date, cast numeric fields, normalize True/False strings, standardize names for launch sites and orbit categories, and validate Class $\in$ {0,1}. Inspect duplicate FlightNumber, unparseable payload values and unusual records. Preserve missing LandingPad as a valid “not recorded / not applicable” state rather than automatically imputing an operational site.

The supplied 90-row file contains **26 blank LandingPad fields**; the other core fields in this extract are populated. Target balance is 60 positive and 30 negative labels. After excluding the target, apply train-only preprocessing (imputation and one-hot encoding) to avoid leakage.

Proposed Python audit

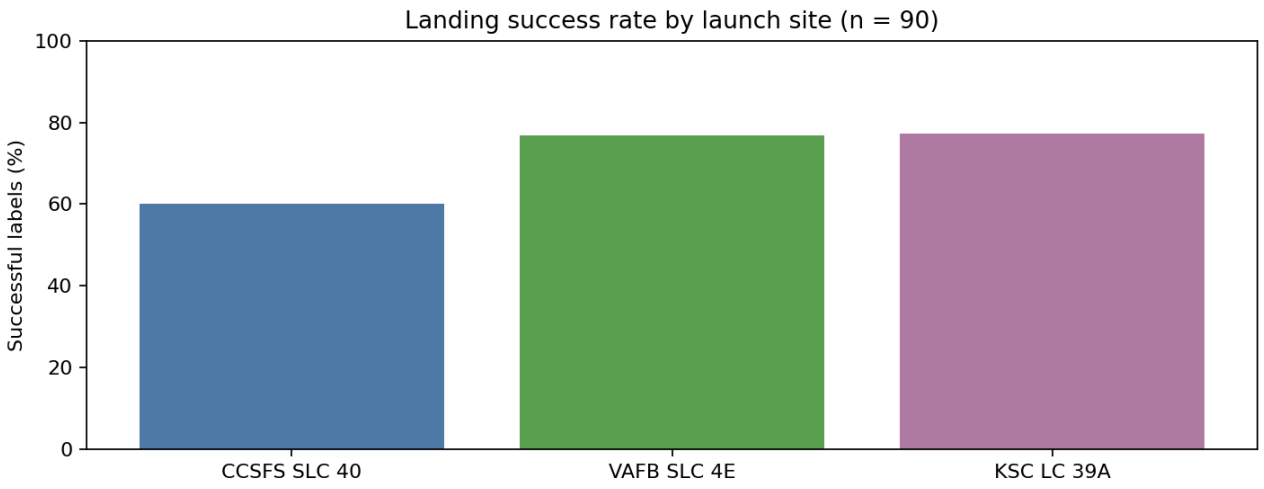
```
df.duplicated(subset=["FlightNumber"]).sum()
df.isna().sum()
df.groupby("LaunchSite")["Class"].agg(["count","sum","mean"])
```

4. Exploratory analysis (EDA): SQL and visualizations

Load the cleaned data into SQLite as launches and query landing rates by location and mission type. SQL aggregates must distinguish successes and total rows; avoid averaging text outcome strings.

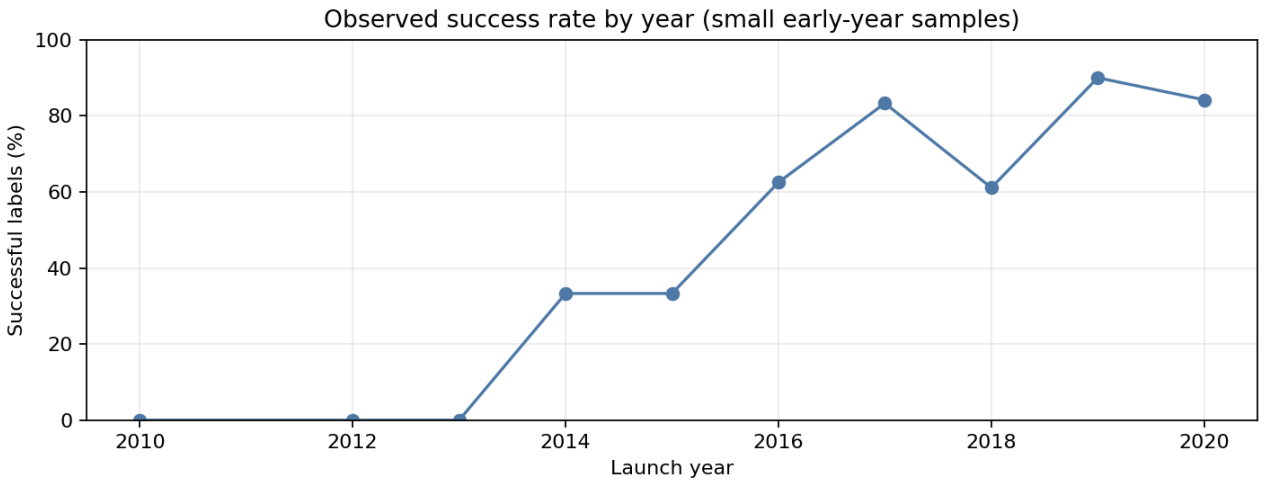
```
SELECT LaunchSite, COUNT(*) AS launches,
SUM(Class) AS successes,
ROUND(100.0 * AVG(Class),1) AS success_pct
FROM launches GROUP BY LaunchSite
ORDER BY launches DESC;
```

Launch site	Launches	Successful	Success rate
CCSFS SLC 40	55	33	60.0%
VAFB SLC 4E	13	10	76.9%
KSC LC 39A	22	17	77.3%



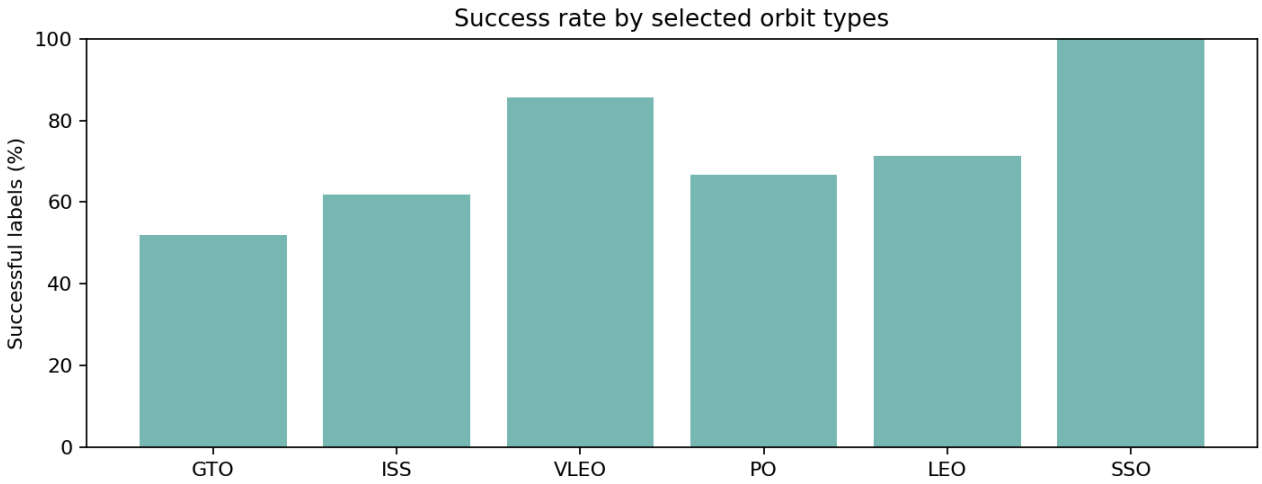
5. EDA: mission and temporal patterns

Success rate by launch year indicates a strong long-run change in the historical labels, but some years have only 1–3 records. Interpret year-to-year changes cautiously and consider temporal confounding from vehicle upgrades and launch mix.



Selected orbit groups

Orbit	Missions	Successes	Success rate
GTO	27	14	51.9%
ISS	21	13	61.9%
VLEO	14	12	85.7%
PO	9	6	66.7%
LEO	7	5	71.4%
SSO	5	5	100.0%



The orbit categories are unevenly represented. Small groups can show 100% success without establishing reliable predictive performance. A scatter plot of PayloadMass by Class and a bar plot of Class by LaunchSite should be included in the executable EDA notebook.

6. Interactive map: Folium design

A Folium map should display site markers at the coordinates in the source dataset: CCSFS SLC 40 (28.5618571, −80.577366); VAFB SLC 4E (34.632093, −120.610829); and KSC LC 39A (Florida). Attach popups with launch counts and success rates. Build MarkerCluster overlays and landing-success filters, then save the map to **launch_site_map.html**.

Measure distances to nearby coastlines, roads, and populated areas using defensible geospatial sources rather than estimating distances by eye. For each launch-site marker, connect location to recorded launches and operational geography.

What is verified here: Site counts and two source-file coordinates. **Not yet verified:** Executed Folium markers, proximity measurements and an exported interactive HTML file.

7. Plotly Dash interactive dashboard

Dashboard layout: (1) a dropdown to select “All Sites” or a launch site; (2) a success/failure pie chart; (3) a payload-mass range slider; (4) a scatter plot of PayloadMass against launch outcome, colored by booster version; (5) a callback that filters every chart on the same selection.

The dashboard should use the supplied spacex_launch_dash.csv or a documented equivalent, and screenshots should show one “All Sites” view and a filtered-site/payload view. Include run command, dependency versions and a reproducible local URL in the repository documentation.

Example usage: python app.py → open localhost:8050. This is an implementation specification, not evidence of a dashboard run.

8. Predictive analysis: multiple classifiers

Compare logistic regression, support vector machine, decision tree and k-nearest neighbors with preprocessing fitted only on training observations. Features can include payload mass, flights, grid fins, booster reuse, legs, Block, ReusedCount, launch site and orbit. Evaluate with a stratified held-out split and five-fold cross-validation on the training set; avoid training on the test set.

Proposed evaluation matrix

Candidate	Cross-validation	Held-out accuracy	Confusion matrix
Majority baseline	Not tuned	66.7% if evaluated on full historical sample; test-set value varies	Required
Logistic regression	GridSearchCV pending	Not measured	Not measured
SVM	GridSearchCV pending	Not measured	Not measured
Decision tree	GridSearchCV pending	Not measured	Not measured
KNN	GridSearchCV pending	Not measured	Not measured

The overall 66.7% is the full-sample majority frequency, not an independently evaluated test score. Report precision, recall, F1 and ROC-AUC where applicable; include positive/negative support and the actual confusion-matrix counts after executing models. A chronological holdout is strongly recommended to assess future-launch generalization.

Do not treat GridFins or Legs as intrinsically causal: correlations may reflect changing operational practice. Some launch features may only be known after a mission; document prediction-time feature availability before production use.

9. Conclusions and limitations

Finding 1: Historical landing success is 66.7% in the sampled 90 launches. **Finding 2:** Site-level success ranges from 60.0% to 77.3%. **Finding 3:** Reused boosters are associated with a higher observed landing-success fraction (29/37 = 78.4%) compared with non-reused boosters (31/53 = 58.5%). **Finding 4:** Grid-fin-equipped records show higher success (58/70 = 82.9%) than records without grid fins (2/20 = 10.0%).

Because the records are historical and observational, these patterns should not be interpreted as causal. The dataset ends in November 2020, and substantial subsequent SpaceX developments are out of scope. Potential reporting conventions, class label definitions, imbalance and training-data leakage can affect all conclusions.

Next actions for a rubric-complete submission: Publish the original, runnable API/web scraping, cleaning, SQL, EDA, Folium, Dash and ML notebooks/scripts to a dedicated GitHub repository; insert verified model metrics and actual app/map screenshots; export these sections as presentation slides; and replace the placeholder GitHub URL in this report.

Data attribution and reproducibility

Source of the 90 records used in the calculations: https://github.com/Abhigyan-RA/SpaceX-capstone/blob/main/dataset_part_2.csv (public educational repository; source author credited). Calculations and charts in this PDF were independently reconstructed from its tabular values. No source notebooks, model scores, or screenshots are represented as the submitter's original execution.

Assignment rubric supplied in "Mark python(1).pdf": 15 subcriteria covering report PDF, GitHub URL, slide PDF, executive summary, introduction, API collection, web scraping, wrangling, EDA and SQL, interactive analytics, Folium, Dash and predictive classification.